

Section 82 — Wavelet Structures And Segment Tree Beats

Two ways ordinary segment trees run out of information

This lesson develops A–C. Problem D is reserved for the editorial.

Problem A: navigate the value domain

A wavelet matrix repeatedly stable-partitions the current sequence by one value bit. For each level store `zero_prefix[i]`, the number of zeros before position `i`, and the total zero count. An interval maps to its zero child as `[pref[l], pref[r])` and to its one child after adding the zero block offset. To find the `k`-th value, count interval zeros, choose the containing side, and subtract the entire zero block when going right.

Coordinate-compress first. Compression preserves order and reduces the number of levels to `ceil(log distinct_values)`.

Problem B: threshold frequencies

The same navigation counts values below an upper-bound rank. Whenever the bound's current bit is one, every zero-side value is certainly smaller and can be added at once; continue only through the one side. This is binary digit DP performed on an interval rather than on one number.

Problem C: segment-tree beats

For range `chmin(x)`, a node can be updated without descending when `second_max < x < max`: only occurrences of the maximum change. Store sum, maximum, second maximum, and maximum multiplicity. Decrease the sum by `(max-x)*count`, then replace the maximum. If `x <= second_max`, different values would change differently and recursion is necessary.

The amortized argument is the algorithm: every expensive descent either crosses a segment boundary or destroys a distinct extremum level. Ineffective updates stop immediately. For two-sided clamps, maintain symmetric minimum information and push both parent bounds.

Wavelet navigation trace

Suppose an interval contains five values whose current bit is 0,1,0,1,0. Its zero count is three. For zero-based $k=3$, the answer is not in the three-zero block, so subtract three and map both interval endpoints through the one-prefix formula. The new k is zero inside the one child.

Every level preserves the relative order within its zero and one partitions. That stability is why an original subarray remains one interval after mapping; an unstable partition would scatter it and destroy the structure.

Threshold trace

When counting values below a bound, a bound bit of one means every interval value taking zero at this level is already smaller regardless of later bits. Add the whole zero-side count and continue in the one side. A bound bit of zero allows no such bulk contribution. This is the same prefix-decision logic used in binary digit DP.

Beats trace and invariant

For multiset values {10,10,7,3}, store maximum 10, second maximum 7, maximum count 2, and the sum. Applying `chmin(8)` changes only the two tens, so subtract $(10-8)*2$. Applying `chmin(6)` also changes 7, so the node summary cannot update all affected values uniformly and recursion is required.

The condition is strict: `second_max < x < max`. When `x==second_max`, the old maximum class merges with the second class and its multiplicity information must be recovered from children.

Implementation and proof habits

Wavelet code should use half-open intervals consistently and decide whether k is zero- or one-based at the API boundary. Coordinate compression preserves ordering but frequency queries involving an external threshold need `lower_bound`, not direct lookup.

For beats, define sentinel second extrema safely, update sum before overwriting the extremum, and push the parent's current cap to children before descending. The proof must state a decreasing potential—destroyed extremum levels—not just claim that recursion “usually stops.”

Tiny oracles sort literal subarrays for wavelet queries and update every array element for beats. Test duplicates, absent thresholds, all-equal ranges, ineffective clamps, and alternating severe min/max operations.

Exercises

A–C correspond to the derivations. D asks you to discover a different “maximum proves no work is needed” pruning rule.